Basic English Underpinning

Architecture, feature mapping, and violation reporting

Technical proposal · Draft 1

Contents

1 Summary 2
2 Motivation 2
3 Feature-to-Basic-English mapping 2
3.1 Parsing to F# and SMT with Z3 discharge 3
3.2 Refiner and fixed-spec/repair gate 3
3.3 Hedgehog generation 3
3.4 Sorts, unification, predicate sorts, constructors, guards, set-operations 3
3.5 Cooperative actors, channels, ChannelEvents, history/rollback 3
3.6 Linear ownership 3
3.7 Cell system: classes, slots, instances, closures, lexical scope 4
3.8 Tensor/image kernel and stack evaluator 4
4 Alternatives and complements 4
5 A two-layer architecture 6
6 Violation reporting 6
6.1 The violation taxonomy 6
6.2 The `Violation` value 7
6.3 Detector-to-feature mapping 8
6.4 Pipeline and control flow 8
7 Prior art 9
8 Testing the checker 10
9 Roadmap 11
10 Non-goals 11
11 Open questions 12
12 Summary of the ask 12

1 Summary

This document proposes how an existing language implementation can be made to underpin Basic English — that is, to **accept**, **interpret**, **execute**, and, crucially, **reject with explanation** sentences in a controlled English whose lexicon is fixed and whose grammar is a small set of licensed constructions.

The implementation already provides a rich set of mechanisms: a tensor/image kernel and stack evaluator with persistence; parsing of Basic English to F# and SMT with Z3 discharge; Hedgehog generation; a refiner and a fixed-spec/repair gate; value sets and sorts with unification, multiple dispatch, predicate sorts and constructors, guards, and set-operation methods; cooperative actors with channels, `ChannelEvent`s, history/rollback, and linear ownership; and an cell-system encoding with closures, lexical scope, classes, slots, and instances.

The proposal has two parts. The first (Section 3) is a **feature-to-role mapping**: an argument that each existing mechanism corresponds to a coherent layer of Basic English semantics, and that the language is already closer to underpinning Basic English than its feature list suggests. The second (Section 6) is the **missing capability**: a first-class, blamable, layer-attributed diagnostics subsystem. A controlled language is only as useful as the explanations it produces when it rejects input; today the pipeline can accept or fail to accept, but it cannot say **which kind** of rule failed, **on which constituent**, or **whether the fault is the user's or the language's**.

The two parts are complementary. The feature mapping establishes that the detectors exist; the violation proposal establishes that they should speak in a shared, actionable vocabulary.

2 Motivation

Basic English is not merely English with a small dictionary. It is a **semantic interface language**: a fixed lexicon (the 850 words and 18 operators), a small and closed set of licensed constructions, and a compositional interpretation that must be **disambiguated**, **verified**, and **executed**. Two consequences follow.

- **Every accepted sentence must be executable**, because Basic English is intended as a general-purpose computer language understandable without training in logic or computing, as Logical English demonstrates.
- **Every rejected sentence must be explained**, because a controlled language whose rejections are opaque will be believed to forbid perfectly ordinary English. With a fixed lexicon, a large fraction of rejections are not grammatical violations but **coverage gaps**; the two must be distinguished.

The document is therefore in two halves. The first establishes that the implementation's mechanisms map cleanly onto the layers of Basic English semantics. The second specifies how each of those mechanisms, when it fails, reports a violation.

3 Feature-to-Basic-English mapping

The implementation's feature groups are not merely sufficient; they are unusually well matched to the layers of a controlled natural language. The mapping below is offered as an argument for that claim, and as a guide to where new work should attach.

3.1 Parsing to F# and SMT with Z3 discharge

Role in Basic English. Logical form and entailment checking. Basic English sentences such as "A man is in the room" can be parsed into typed first-order or SMT formulas. Z3 can then check **consistency**, **entailment**, and **presupposition** — for example, whether "Every student owns a book" entails "Some book is owned by a student." This is the pipeline that Logical English demonstrates: controlled English → logic program → execution and verification. The SMT backend is particularly strong for **quantifier reasoning** (all, some, no) and **sortal constraints** (mass vs. count nouns).

3.2 Refiner and fixed-spec/repair gate

Role. Enforcing the controlled grammar and repairing violations. Basic English has a fixed 850-word lexicon and a small operator set. The refiner rejects out-of-vocabulary words and unlicensed constructions; the repair gate proposes the nearest Basic English paraphrase ("commence" → "start"). This pair is the acceptance criterion for a **closed-world** controlled language in which every accepted sentence must be executable.

3.3 Hedgehog generation

Role. Property-based testing of the grammar and the semantics. Basic English sentences can be generated from the grammar and checked against universal properties: **compositionality** (the meaning of a sentence is a function of its parts), **sort preservation** (subjects match verb sorts), and **idempotence of repair** (repairing a repaired sentence changes nothing). Hedgehog is particularly suitable because its integrated shrinking produces minimal counterexamples in the grammar.

3.4 Sorts, unification, predicate sorts, constructors, guards, set-operations

Role. The semantic type system and compositional semantics.

- **Sorts** model semantic categories: `Human`, `PhysicalObject`, `Event`, `Proposition`, `MassNoun`, `CountNoun`.
- **Predicate sorts and constructors** model verb frames and prepositional meanings (e.g., `in : Object × Location → Proposition`).
- **Unification** handles **agreement** (subject–verb number) and **anaphora resolution** (binding "he" to the most salient male entity).
- **Guards** implement **selectional restrictions** (the subject of "dream" must be animate) and **conditional sentences**.
- **Set-operation methods** directly support **quantifiers and plurals**: "all men" → `∀x:Man. …`, "some boys" → `∃x:Boy. …`, "no women" → `¬∃x:Woman. …`, "most" → cardinality constraints.

3.5 Cooperative actors, channels, ChannelEvents, history/rollback

Role. Discourse and dialogue management. Basic English is often used for instructions and interactive dialogue. Actors represent speaker/hearer agents or discourse participants. Channels carry utterances; `ChannelEvent`s record speech acts (assertion, question, command). History/rollback permits context-sensitive interpretation and repair when a later sentence contradicts an earlier one. The ParseTalk model uses exactly this actor paradigm for concurrent natural-language parsing, with asynchronous message passing between lexical and grammatical actors.

3.6 Linear ownership

Role. Resource-sensitive semantics and movement. Linear logic has long been used to model resource sensitivity in natural language — for example, the fact that a pronoun consumes its

antecedent once, or that a quantified phrase cannot be reused. Linear ownership can enforce single-use constraints on discourse referents, preventing "he" from referring to two different men in the same scope without explicit accommodation.

3.7 Cell system: classes, slots, instances, closures, lexical scope

Role. Lexical semantics and ontology. A **class** represents a lexical concept (`Dog`, `Run`, `Red`); **slots** represent selectional preferences and lexical relations; **instances** represent specific referents in a discourse model. Closures and lexical scope model **binding** in quantified phrases and relative clauses ("the man who saw every woman"). The cell system is a natural home for a toy ontology aligned with the 850-word lexicon.

3.8 Tensor/image kernel and stack evaluator

Role. Grounding and incremental interpretation. The tensor kernel is the least directly linguistic mechanism, but it can serve **perceptual grounding** when Basic English is used to describe images or scenes. The stack evaluator supports **incremental parsing** and **stack-based discourse updates** (push a new referent, pop on topic shift).

4 Alternatives and complements

The implementation is already well equipped. The table below records, feature by feature, where its current form has a limit for Basic English, and what the Racket/CLOS community and the broader language-design community offer as an alternative or complement.

Feature	**Limit for Basic English**	**Alternative / complement**
Multiple dispatch (CLOS-style)	Verb meanings are highly **polysemous**; type-based dispatch can be too coarse.	**Predicate dispatching** (a CLOS extension) dispatches on arbitrary predicates: `(defmethod interpret ((v run) (subj animated)) …)` vs. `(defmethod interpret ((v run) (subj machine)) …)`. Racket's `racket/class` with **mixins** composes semantic modules — a "motion-verb" mixin — without a fixed class hierarchy.
Cooperative actors	Messages can become unbounded in open-domain dialogue.	Racket's `racket/actor` gives lightweight actors with thread-safe channels. Erlang/OTP processes with selective receive add supervision and fault tolerance, which dialogue systems need.
Linear ownership	Strict linearity is too rigid: natural language **weakens** (drops a pronoun) and **contracts** freely.	Linear Haskell and Rust's ownership offer practical approximations. For linguistics, **abstract categorial grammars** already use linear λ-calculus as the semantic backbone; an ACG-style linear type system would be more principled than general ownership.
Unification	First-order unification cannot handle **scope ambiguity** or **ellipsis**.	Higher-order unification (as in λ-Prolog) or miniKanren-style logic programming in Racket handles scope and ellipsis more gracefully.
SMT / Z3	Excellent for **quantifier reasoning**, weak for **defaults** and **non-monotonicity**.	Pair Z3 with Answer Set Programming (e.g., `clingo`) for defeasible reasoning, as Logical English does with LPS.
Cell system	A class-based ontology can become rigid for lexical creativity.	Prototype-based systems (Self, JavaScript) or CLOS with method combination permit **aspect-oriented** lexical semantics: primary methods for core meaning, `:before`/`:after` for coercions and pragmatics.
Hedgehog	Property-based testing of a grammar requires **semantic oracles**.	QuickCheck with custom generators, or Hypothesis with `hypothesis-grammar`. For semantic testing, **property-based translation validation** (parse → SMT → model) is more direct.
Stack evaluator.	A pure stack evaluator struggles with **long-distance dependencies**.	Continuation-passing style, or delimited continuations (Racket's `racket/control`), model filler–gap dependencies and scope extrusion more naturally.

Table 1: Alternatives and complements, with the Racket/CLOS community emphasized.

The single most valuable addition from this table for Basic English is **predicate dispatching with method combination**. English syntax is productive through **constructions** (caused motion, resultative, ditransitive) rather than through enumerated verb classes; predicate dispatch lets the semantics attach to the construction rather than to each lexical entry. Racket's `racket/generic` provides generic methods, and the Swindle library brings CLOS-style multimethods and method combination to Racket.

5 A two-layer architecture

For Basic English, the feature mapping suggests a two-layer architecture that separates **controlled surface** from **semantic interpretation**.

1. **Controlled surface layer.** The refiner and repair gate enforce the 850-word lexicon and the 18 operators. The parser produces a **typed abstract syntax tree** using the sort system and predicate constructors. This layer owns admission and rejection.
2. **Semantic layer.** The AST is translated to SMT-LIB for verification (Z3) and to logic programs (Prolog/Datalog) for execution. The actor layer manages discourse state and dialogue moves. Linear ownership tracks discourse referents across turns.

This mirrors the design of Logical English, which is explicitly intended as a general-purpose computer language understandable without training in logic or computing. It also has the virtue of making the diagnostics requirement structural: each layer has its own detectors, and each detector has its own explanatory vocabulary.

6 Violation reporting

"Violates the grammar" is not one condition. A controlled language with a fixed lexicon must distinguish several failure modes, and the design of the reporting subsystem follows from that distinction.

6.1 The violation taxonomy

Violations are classified by the pipeline stage that detects them. This classification is user-facing: the reported `layer` selects which repair strategies and which explanatory prose are available.

Layer	**Example**	**Detector**	**Explains itself as**
Lexical	`"commence"`	Refiner + fixed spec	word not in 850-word list; nearest legal word
Structural	`"the man the dog"`	Parser / refiner	not a licensed construction; expected *NP VP*
Agreement	`"the men is here"`	Unification	number mismatch: subject plural, verb singular
Sortal	`"the stone dreams"`	Guards, predicate sorts, dispatch	subject of `dream` must be `Animate`, given `PhysicalObject`
Binding	`"he left"` (no antecedent)	Linear ownership, closures	unbound pronoun; referent already consumed
Semantic	`all men are mortal` $\wedge$ `¬mortal(Socrates)`	SMT $\rightarrow$ Z3	unsatisfiable; unsat core is these assertions
Discourse	contradicts an earlier utterance	Actors + history	contradicts utterance 3; rolled back to state 2

Table 2: Seven violation layers, each with its own detector and its own explanatory vocabulary.

The layer distinction is **load-bearing** for the user interface. Agreement and binding violations are local and admit a deterministic repair; sortal violations admit several candidate repairs and must be ranked; semantic and discourse violations are witnessed by a model or an unsat core, not by an edit.

Discourse boundary. The `Discourse` layer is a host-provided contradiction; `Discourse.utter` implements the rollback policy, not a contradiction detector. No production detector emits this layer yet. Likewise, `NearestSentence` is sentence advice, while `FixedSpecificationAcceptance` is the fixed-spec implementation gate; they are separate APIs.

Decision (D79): The showcase and the CLI `--diagnose` mode render a parser violation, a nearest suggestion, a discourse rejection, and an SMT counterexample in the same sink (D76–D78). The fixed-spec gate does not call sentence search: `ImplementationProposal` cannot carry a specification edit, `Violation.Suggestion` is advice, and `SpecificationProposal` requires an exploratory capability.

6.2 The `Violation` value

All detectors produce the same record. Fields are optional only when the layer makes them meaningless, never for convenience.

```
struct Violation {
    layer:    Layer,          // Lexical | Structural | Agreement | Sortal
                              // | Binding | Semantic | Discourse
    rule:     RuleId,         // identifier in the fixed spec
    span:     Span,           // source span of the offending constituent
    expected: Option<Term>,   // the guard / sort / contract that failed
    given:    Option<Term>,   // the offending term, if refined
    blame:    Blame,          // Input | CoverageGap
    witness:  Option<Witness>,// UnsatCore | Model
```

```
    repair:   Option<String>, // nearest legal Basic English, if any
}
```

Two fields deserve comment.

`blame`. Following `racket/contract`, every failure names a party. Here the two parties are the **input** (the user wrote something Basic English forbids) and the **language** (the user wrote something the fixed spec does not cover). The distinction changes what the checker should tell the user: in the first case a repair; in the second, a note that this is a coverage gap, optionally with the closest legal approximation. Without this field, the two cases are indistinguishable to the user, and the checker becomes frustrating.

`witness`. For semantic and discourse layers, a boolean is nearly useless. The useful artefact is the Z3 `unsat_core()` (which assertions conflict) or a satisfying model (a concrete counterexample to the user's expectation). Both are cheap to produce given that the SMT backend is already in the pipeline.

6.3 Detector-to-feature mapping

Each existing feature is a violation detector with a diagnostic obligation. The proposal is to give each one a `Violation`-shaped output and route it to a common sink.

Existing feature	**Violation role**
Refiner + fixed-spec gate	Primary emitter for `Lexical` and `Structural`. The gate **is** the acceptance criterion; the repair gate is its dual.
SMT + Z3	`Semantic`. Report `unsat_core()` as the witness; report the model for satisfiable-but-suspicious cases. Per-rule queries or `assert-soft` permit multiple simultaneous reports.
Value sets, sorts, predicate sorts, constructors	`Sortal`. Sort constructors carry a `sort-name`, which is what the diagnostic prints. A failed guard reports its own identity.
Unification	`Agreement`. Report the conflicting feature values, not just "fail".
Multiple dispatch	`Sortal` at the lexical-frame level: "no method applies" is exactly "no rule licenses this verb–argument combination".
Linear ownership	`Binding`. Reports resource violations that are otherwise invisible: referent consumed twice, pronoun without antecedent.
Cooperative actors + `ChannelEvents` + history/ rollback	Diagnostic **transport** and **transactionality**. Violations are published as events; a violation triggers rollback so a bad utterance does not commit discourse state.
Closures, lexical scope, cell system	Model of **binding** and **ontology**; provides the terms that `expected` and `given` refer to.
Hedgehog	Testing the checker itself (see Section 8).

Table 3: Each existing subsystem, viewed as a violation detector with a diagnostic obligation.

6.4 Pipeline and control flow

The `Violation` value is the ordinary currency between stages; the diagnostic channel is the observable side-channel.

Input → Refiner → Parser → Sort & Guard → Unification → SMT → Discourse

each stage, on failure, emits `Violation` to the **diagnostic channel**;
the **repair gate** consumes the accumulated violations and proposes a minimal edit;
the **discourse actor** rolls back to the previous committed state if any `Blame::Input` violation at or above the `Structural` layer occurred.

Three properties are worth calling out.

- **Localized detection, centralized reporting.** Each detector knows only its own rule; the sink knows how to merge, rank, and present. This avoids the temptation to encode presentational logic in each checker.
- **Blame-conservative acceptance.** A `CoverageGap` should not, by default, block execution; an `Input` violation should. The refiner's policy is a single decision point, not scattered across stages.
- **Transactional discourse.** Because violations arrive via actors and `ChannelEvent`s, a bad utterance is a **rollback event**. The history mechanism already in place makes this an integration, not new machinery.

7 Prior art

The Racket and CLOS ecosystems, and the controlled-natural-language literature, provide unusually good models for the violation subsystem. The table records the specific idea taken from each source.

Source	Borrowed idea
`racket/contract`	Positive/negative blame; `exn:fail:contract` as a structured, blame-attributed failure. Direct model for `Blame`.
`syntax-parse`	Declarative grammar rules with `#:fail-when`, `#:fail-unless`, `#:error`, `#:context`; the "expected X, given Y, at span Z" idiom.
`redex`	`define-language` plus `redex-check`: grammar-admission testing with automatic counterexample generation.
`turnstile` / `turnstile+`	Custom type errors raised from within the type system; the principle that unification should never surface as a bare "fail".
CLOS method combination, `no-applicable-method`	First-class "I cannot interpret this" signals; predicate dispatch failures that name the failed predicate.
Grammatical Framework (`gf -check`)	Separated abstract and concrete syntax with type-checked abstract trees; the model of "ill-typed tree" as a first-class diagnostic.
Attribute grammars (JastAdd, Silver)	Errors attached to productions with precise source locations.
Answer Set Programming (`clingo`)	Defeasible rules, unsat cores, and explanations; complements SMT's weakness on defaults and non-monotonicity.
Datalog with provenance	Why-provenance: "this violation fired because of rules R on terms A, B".
ANTLR/Xtext error listeners	Engineering pattern for continuing past the first error and reporting all errors in one pass, with spans.

Table 4: Prior art and the specific idea taken from each source.

8 Testing the checker

A violation reporter is only useful if it fires with the **right** rule id and layer. Property-based testing is the natural fit and is already in the pipeline via Hedgehog.

- **Grammar → acceptance.** Hedgehog generators produce sentences from the grammar; the property asserts the validator accepts them with no `Violation`.
- **Near-miss → rejection.** Generators produce minimal perturbations (agreement flips, sort substitutions, dropped antecedents); the property asserts the validator rejects them with the **expected** `rule` and `layer`. Hedgehog's shrinking produces the **minimal violating sentence** — the best possible error message.
- **Repair idempotence.** Property: `repair(repair(s)) == repair(s)` for every generated `s`, and the repair itself is accepted.
- **Discourse transactionality.** Property: after a rejected utterance, the discourse state equals the state before it.

`redex`'s `redex-check` provides the same loop within Racket and is worth using in parallel: `define-language` gives a second, independent grammar model whose admission decisions can be cross-checked against the main refiner.

9 Roadmap

Phase	Deliverable	Acceptance
P0 ✓	`Violation` type, diagnostic channel, refiner and parser emit it	All `Lexical` and `Structural` failures carry rule, span, blame
P1 ✓	Unification, sorts, guards, dispatch emit `Violation`	Agreement and sortal failures name the offending term and the failed rule
P2 ✓	SMT integration: `unsat_core` as witness; the model as `Given`; per-rule queries for multi-report	Semantic violations reported with core; no dropped violations
P3 ✓	Discourse layer publishes violations; rollback on `Blame::Input`	State equality after rejected utterance; Hedgehog property passes
P4 ✓	Minimum-edit repair search over the fixed grammar	Idempotent; repair accepted; ranked candidates for `Sortal`
P5 ✓	Presentation layer: layer-aware rendering, expected/given, source span	Usable output for a non-logician Basic English user

Table 5: Phased roadmap. Each phase is independently shippable and testable.

Status. Phases P0, P1, P2, P3, P4, and P5 are applied as D57, D61, D62, D70, D71, and D72. `violations.fs` gives every grammar refusal and every semantic failure a layer, rule, span, expected/given, blame (`Input` vs `CoverageGap`), and an optional repair; `Violations.parse` returns `Result` and `Violations.ofEvaluationError`/`ofGuardVerdict` cover the evaluator. The SMT stage reports through `Smt.dischargeWithCore` and `Violations.ofSmtDischarge`: a falsified obligation carries the solver model, a proved obligation or a refuted possibility carries the labelled unsat core, and `Unknown`/`Unavailable` stay `CoverageGap`. P3 is applied as D70: `Discourse.utter` publishes the violation and restores the pre-utterance state on an input fault at `Structural` or above; a coverage gap does not block. P4 is applied as D71: `RepairSearch.search` finds the nearest accepted token list under the fixed grammar and `searchWith` ranks candidates through a layer check. P5 is applied as D72: `Presentation.render` gives the layer-aware prose, expected/given, span, evidence, and repair; `renderTechnical` keeps the rule id. The span is the offending canonical token. Character offsets remain open.

10 Non-goals

- Full natural-language parsing outside the fixed Basic English spec. The refiner's role is explicitly to reject out-of-scope input; the proposal is about **how** it rejects.
- A general proof-of-unsatisfiability translator. P5 renders the layer, span, expected/given, evidence, and repair; it does not translate a Z3 proof into prose.
- Replacement of the existing repair strategies. The proposal standardizes their input and output; it does not redesign them.

11 Open questions

- **Granularity of `rule`.** Should `RuleId` be per-production, per-construction, or per-lexical-entry? The `syntax-parse` model suggests per-production; the Basic English operator count (18) suggests per-operator is also tractable.
- **Ranking repairs for `Sortal`.** Minimum-edit is underdetermined when several sorts can be substituted. Do we prefer shorter edits, edits that preserve the intended sort, or edits that preserve the verb? A small preference model will be needed.
- **Default-on or default-off for `CoverageGap`.** Should an out-of-vocabulary word block execution (strict Basic English) or pass through with a warning (pragmatic use)? This is a policy decision the refiner must expose as a parameter, not a constant.
- **Cross-layer dependencies.** An agreement violation may mask a sortal violation that would only be visible after the agreement is repaired. Do we report both (requiring continued checking past the first failure) or only the first? The ANTLR/Xtext pattern favors both; the cost is more complex recovery.
- **Dialect of `Blame`.** `racket/contract` uses positive/negative parties; the natural analogue here is `Input`/`Coverage`. A third case, `Internal` (a bug in the checker itself), should probably exist but is out of scope for a first version.

12 Summary of the ask

The proposal has two parts, corresponding to what the implementation already does and to what it must be able to say.

- **Architecture.** Each existing feature is already the detector for a coherent layer of Basic English semantics: parsing and SMT for logic and entailment; the refiner and repair gate for the fixed lexicon; sorts, guards, unification, and dispatch for the type system and compositional semantics; actors and history for discourse; linear ownership for resource-sensitive binding; the cell system for the ontology; Hedgehog for testing. The recommended complement is **predicate dispatching with method combination**, which lets the semantics attach to constructions rather than to enumerated verb classes.
- **Violation reporting.** One new type (`Violation`), one new channel (the diagnostic channel, carried by existing actors), one new policy point (blame-based acceptance in the refiner), and the repair gate as its consumer. It removes nothing and replaces nothing; it makes each existing feature say **why** it failed, in a form a user can act on.